

Architecture et Programmation Web

Séance 2 : Introduction approfondie à Flask

Nicolas Burny

6 mars 2026

ECAM

Introduction et Concept

Routage

Traiter les Requêtes avec Flask

Construction d'URLs

Flux de Contrôle

Organisation des fichiers

Introduction et Concept

- **Minimalisme** : Ne fournit que le routage et le templating.
- **Extensibilité** : Vous gardez le contrôle sur chaque brique (Base de données, Auth).
- **Simplicité** : Pas de structures imposées lourdes.

Flask repose sur deux bibliothèques majeures :

1. **Werkzeug** : Pour le protocole WSGI et le routage.
2. **Jinja2** : Pour le moteur de templates (vu en Séance 3).

WSGI

Interface standard entre les serveurs Web et les applications Python.

Application Minimale

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello world!"

if __name__ == '__main__':
    app.run(debug=True)
```

Pourquoi debug=True ?

- **Auto-reload** : Le serveur redémarre à chaque modification du code.
- **Debugger interactif** : Affiche une console dans le navigateur en cas d'erreur.
- **Danger** : Ne jamais activer en production !

Routage

Dans une application web, le **routage** consiste à faire correspondre une URL entrante à une action spécifique côté serveur.

- L'utilisateur demande une ressource via son navigateur (ex : /contact).
- Le serveur analyse l'URL demandée et la méthode HTTP (GET, POST, etc.).
- Il « route » la requête vers le bloc de code (fonction) chargé de traiter cette demande précise.

Qu'est-ce qu'une Route ?

C'est le lien entre une **URL** et une **Fonction Python** (View Function).

- Utilisez le décorateur `@app.route()`.
- Une fonction peut avoir plusieurs routes.

```
@app.route('/contact')  
def contact():  
    return "Contactez-nous à support@example.com"
```

On peut capturer des segments de l'URL dans des variables :

```
@app.route('/user/<username>')  
def show_user(username):  
    return f"Profil de {username}"
```

On peut forcer le type de donnée attendu :

- `string` : Texte (par défaut).
- `int` : Entier.
- `float` : Nombre décimal.
- `path` : Comme `string` mais accepte les slashes.
- `uuid` : Identifiants uniques.

```
@app.route('/post/<int:post_id>')  
def show_post(post_id):  
    # post_id est garanti d'être un entier  
    return f"Article numéro {post_id}"
```

Traiter les Requêtes avec Flask

Chaque échange Web commence par une requête structurée :

- **Protocole** : HTTP ou HTTPS.
- **URL** : L'adresse complète (Host + Path).
- **Méthode** : L'action souhaitée (GET, POST...).

Exemple : `https://www.monsite.com/blog/article-1`

Il existe plusieurs façons de passer de l'information :

- **Path Parameters** : Inclus dans le chemin (/user/12).
- **Query Parameters** : Après le ? (/search?q=flask).
- **Headers** : Métadonnées techniques (User-Agent, Content-Type).
- **Body** : Données transmises dans le corps de la requête (POST).

L'objet Request de Flask¹

Accéder aux données envoyées par le client :

- **GET parameters** : `request.args`.
- **Headers** : `request.headers`.
- **Method** : `request.method`.
- **Body** : `request.data`, `request.form` or `request.json`.

1. <https://stackoverflow.com/questions/10434599/get-the-data-received-in-a-flask-request>

Gestion des Méthodes HTTP

Par défaut, une route ne répond qu'au GET.

```
from flask import request

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        # Extraction des données du dictionnaire request.form
        username = request.form.get('username')
        password = request.form.get('password')

        if username == "admin" and password == "secret":
            return f"Bienvenue {username}, vous êtes connecté !"
        return "Identifiants invalides", 401
```

Construction d'URLs

Ne jamais écrire d'URLs en dur dans votre code HTML ou Python.

- `url_for('nom_de_la_fonction')` génère l'URL automatiquement.
- Si vous changez l'URL (`/login` → `/connexion`), aucun lien ne sera cassé.
- Possibilité de générer des URLs avec des paramètres

```
from flask import url_for

@app.route('/')
def index():
    return f'Voir le profil de \
    <a href="{url_for("show_user", username="nicolas")}">Nicolas</a>'

@app.route('/user/<username>')
def show_user(username):
    return f"Profil de {username}"
```

Flux de Contrôle

Une redirection est une action permettant de rediriger un utilisateur vers une autre page.

- Utile après une action réussie (ex : après un post de formulaire).
- **Exemple** : `return redirect(url_for('home'))`.

Flask permet de gérer les erreurs et d'interrompre proprement une requête :

- `abort(404)` : Page non trouvée.
- `abort(401)` : Accès refusé.

Il y a d'autres erreurs comme 500 (Internal Server Error), 403 (Forbidden), etc.

Voir : <https://developer.mozilla.org/fr/docs/Web/HTTP/Status>

```
@app.errorhandler(404)
def page_not_found(error):
    return "Cette ressource n'existe pas.", 404
```

Organisation des fichiers

Les fichiers statiques sont des ressources qui ne sont pas générées dynamiquement (CSS, JavaScript, images, polices).

- **Emplacement par défaut** : Flask cherche automatiquement dans un dossier nommé `static/` situé à la racine du projet.
- **Structure recommandée** :
 - `static/css/style.css`
 - `static/js/script.js`
 - `static/img/logo.png`
- **Service automatique** : En mode développement, Flask sert ces fichiers via la route `/static`.

Lier des Fichiers Statiques

Pour garantir que les liens fonctionnent quel que soit le déploiement, utilisez toujours `url_for('static', filename='...')`.

Dans un template HTML (Jinja2) :

```
<!-- Inclusion du CSS -->
<link rel="stylesheet"
      href="{{ url_for('static', filename='css/style.css') }}">

<!-- Affichage d'une image -->

```

Pourquoi ? Cela gère automatiquement les préfixes d'URL et assure la portabilité de votre application.

Questions ?